

UAI301 Object-Oriented Programming

Lab Assignment 5.1

Topic: Encapsulation (Access Specifiers - Public, Private, Protected, Name Mangling, Data Hiding and Abstraction)

- 1) Write a Python program:

```
class Employee:
    def __init__(self):
        self.name = "Ravi"
        self._department = "CSE"
        self.__salary = 50000
```

Create an object and perform:

```
emp.name = "Amit"
emp._department = "ECE"
```

Then try:

```
emp.__salary = 70000
```

Print all three values.

Finally, use name mangling to print the original salary.

*Question: Does `emp.__salary = 70000` modify the original private variable?

- 2) Write a Python program to create a Patient class with:

name	# public
_age	# protected
__disease	# private

Create an object and display:

Patient name

Patient age

Patient disease using name mangling

Also try to access `__disease` directly and observe the error.

- 3) Write and execute the following program:

Answer the following:

- I. What is the output?
- II. Why are the two values different?
- III. Did `emp.__salary = 80000` change the original private variable?

```
class Employee:
    def __init__(self):
        self.__salary = 50000

emp = Employee()
emp.__salary = 80000
print(emp.__salary)
print(emp._Employee__salary)
```

3. Create a DigitalWallet class with:

Implement methods:

- deposit(amount)
- withdraw(amount, security_code)
- check_balance()
- change_security_code()

owner_name	→ public
_wallet_type	→ protected
__balance	→ private
__security_code	→ private

Additional requirement

If the withdrawal amount is greater than the balance, display an appropriate message.

The program should ensure that the balance and security code cannot normally be manipulated directly from the main program.

4. Design a SmartDoor class for a smart home. Use:

Implement:

```
lock_door()
unlock_door(pin)
change_pin(old_pin, new_pin)
show_status()
```

owner_name	→ public
_door_status	→ protected
__security_pin	→ private

Requirements

- Door status should initially be "Locked".
- The door should unlock only with the correct PIN.
- The PIN should never be printed.
- The user should be able to change the PIN only after providing the old PIN.
- Use a private variable for the PIN.

CASE STUDIES

Case Study 1: E-Commerce Order Processing (Protected Logistics)

An e-commerce platform manages shipments using a base class Order and a specialized subclass ExpressOrder. Shipping routing information needs to be shared within the shipping pipeline (child classes) but hidden from public API consumers.

Write a Python program demonstrating protected members in inheritance:

- Create a parent class Order with a protected attribute `_shipping_status` initialized to "Processing".
- Create a subclass ExpressOrder that inherits from Order.
- In ExpressOrder, write a method `expedite_shipping()` that modifies `_shipping_status` to "In Transit - Priority".
- Implement a public `track_order()` method in ExpressOrder that returns the status.
- Demonstrate instantiating an ExpressOrder, calling `expedite_shipping()`, and tracking the result.

Case Study 2: Ride-Hailing App (Taxi Driver Privacy)

A ride-sharing app allows riders to view driver details (driver name, car location) but strictly hides sensitive driver information (phone number/contact) behind internal methods.

Write a Python program for a Taxi class:

- Attributes: driver (public), _location (protected), and __contact (private).
- Write a public method disp() that displays all three attributes in a single line.
- Demonstrate what happens when an external caller attempts to print taxi1.__contact directly versus accessing it via disp().
- Show how an admin can still access __contact outside the class using Python's **Name Mangling** syntax (_Taxi__contact).
- Test updating the patient's age with both a valid value (35) and an invalid value (150).

Case Study 3: Gaming Engine (Player Health & Invincibility)

In an action RPG game, player health (__hp) must be strictly controlled through damage and healing functions to prevent cheat scripts from setting health to arbitrary numbers.

Write a Python program:

- Create a Player class with public username and private __hp (default 100).
- Add a public method take_damage(amount) that reduces __hp. Ensure __hp cannot drop below 0.
- Add a public method heal(amount) that increases __hp. Ensure __hp cannot exceed 100.
- Write a method is_alive() that returns True if __hp > 0, otherwise False.
- Simulate a combat sequence where a player takes damage, heals, and attempts direct player.__hp = 9999 manipulation.